

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Actuator, Code Structure, Configuration Classes

Table of Contents

Learning Outcome:	3
1. Actuator	4
Overview of Actuator	4
Capabilities of Actuator	4
Actuator Endpoints	5
Custom Actuators	5
2. Spring Boot Code Structure	6
Default Package	6
Typical Layout:	6
3. Importing Additional Configuration Classes	7
Overview of Configuration Classes	7
Importing Additional Configuration Classes	7
4. Spring Beans and Dependency Injection	9
Overview of Spring Beans	9
Dependency Injection	9
Constructor Injection	9
Setter Injection	10
Field Injection	11
Types of Dependency Injection	11

UNIT 3: Actuator, Code Structure, Configuration Classes

Objectives:

In this Unit you will learn to –

- Understand the capabilities of Actuator and its endpoints.
- Implement custom actuators in your Spring Boot application.
- Understand the Spring Boot code structure and its typical layout.
- Import additional configuration classes and XML configuration.
- Understand the basics of Spring Beans and dependency injection.

Learning Outcome:

Understand Spring Boot's actuator, code structure and configuration classes.

3. Actuator, Code Structure, Configuration Classes

In this unit, you will learn about Actuator, its capabilities, endpoints, and custom actuators. You will also learn about Spring Boot code structure, default package, typical layout, importing additional configuration classes, importing XML configuration, Spring Beans, and dependency injection.

1. Actuator

Overview of Actuator

Actuator is a sub-project of Spring Boot that provides several endpoints and features to monitor and manage your application's health and status. It allows you to monitor various aspects of your application like health, metrics, info, trace, and more. Actuator provides various endpoints that can be accessed over HTTP or JMX, and it also provides various customizable features to extend and configure the endpoints as per the application's requirements.

Capabilities of Actuator

The following are the capabilities of Actuator:

- **Health:** Actuator provides a /health endpoint that returns the current health status of your application. This endpoint can be used by monitoring tools to determine whether the application is running smoothly or not.
- **Metrics:** Actuator provides a /metrics endpoint that returns various metrics related to your application, such as memory usage, CPU usage, and request count, among others.
- **Info:** Actuator provides an /info endpoint that returns general information about your application, such as the application name, version, and description.
- **Trace:** Actuator provides a /trace endpoint that returns the recent HTTP requests and responses.

- **Auditing:** Actuator provides an auditing endpoint to keep track of the changes in the application's state.
- **Environment:** Actuator provides an `/env` endpoint that returns information about the application environment, such as system properties and environment variables.

Actuator Endpoints

Actuator provides various endpoints that can be accessed over HTTP or JMX. The following are the endpoints provided by Actuator:

- **/health:** Returns the health status of your application.
- **/metrics:** Returns various metrics related to your application.
- **/info:** Returns general information about your application.
- **/trace:** Returns the recent HTTP requests and responses.
- **/auditevents:** Returns a list of audit events.
- **/loggers:** Returns a list of loggers and their current configuration.
- **/heapdump:** Returns a heap dump of the application.
- **/threaddump:** Returns a thread dump of the application.

Custom Actuators

Actuator provides various customizable features to extend and configure the endpoints as per the application's requirements. You can create custom endpoints by implementing the `ActuatorEndpoint` interface or by using the `@Endpoint` annotation. You can also add custom health indicators to monitor specific aspects of your application's health by implementing the `HealthIndicator` interface. You can also customize the existing endpoints by using the Actuator's configuration properties.

In summary, Actuator provides various endpoints and features to monitor and

manage your application's health and status. It allows you to monitor various aspects of your application like health, metrics, info, trace, and more. Actuator provides various customizable features to extend and configure the endpoints as per the application's requirements.

2. Spring Boot Code Structure

Default Package

In a Spring Boot application, the default package is the root package of your application. It is the package in which you have placed your main class that is annotated with `@SpringBootApplication`. The default package name is used as a prefix for component scanning, which means that Spring Boot will scan all the packages and sub-packages under the default package for components and beans.

Typical Layout:

A typical Spring Boot application has a standard directory structure that organizes your code and resources in a predictable way. The following is the typical directory structure of a Spring Boot application:

- **src/main/java:** Contains the Java source code for your application.
- **src/main/resources:** Contains the application's resources, such as configuration files and static resources.
- **src/test/java:** Contains the Java source code for your application's tests.
- **src/test/resources:** Contains the resources for your application's tests.

Within the `src/main/java` directory, the Java source code is organized into packages that represent the different modules or components of your application. Each package typically contains Java classes and interfaces that define the components and services of the module.

The `src/main/resources` directory contains the application's configuration files and

static resources. The configuration files include `application.properties` or `application.yml` files that specify the application's properties and settings. The static resources include files such as HTML, CSS, and JavaScript files.

The `src/test/java` directory contains the Java source code for your application's tests. The tests are typically organized in the same package structure as the main source code.

The `src/test/resources` directory contains the resources for your application's tests. The resources include files such as test configuration files and test data files.

In summary, a Spring Boot application has a standard directory structure that organizes your code and resources in a predictable way. The default package is the root package of your application, and it is used as a prefix for component scanning. The typical directory structure includes `src/main/java`, `src/main/resources`, `src/test/java`, and `src/test/resources` directories. The Java source code is organized into packages that represent the different modules or components of your application, and the resources include configuration files and static resources.

3. Importing Additional Configuration Classes

Overview of Configuration Classes

Configuration classes are special classes in a Spring Boot application that are used to define the configuration of the application. These classes typically contain bean definitions, which are used to instantiate and configure beans for the application. Configuration classes are annotated with the `@Configuration` annotation, and the bean definitions are annotated with the `@Bean` annotation.

Importing Additional Configuration Classes

In a Spring Boot application, you can import additional configuration classes to define the configuration of your application. You can import a configuration class by using the `@Import` annotation on the main configuration class or on any other configuration class. When you import a configuration class, all the beans defined in

that class will be available in the application context.

For example, suppose you have a configuration class named `MyConfig` that defines a bean named `myBean`. You can import this configuration class into your main configuration class by using the `@Import` annotation as follows:

```
@Configuration
@Import(MyConfig.class)
public class MyAppConfig {
    // ...
}
```

In this example, the `MyConfig` class is imported into the `MyAppConfig` class, and the `myBean` bean defined in the `MyConfig` class will be available in the application context.

Importing XML Configuration

Spring Boot also supports importing XML configuration files in addition to Java configuration classes. To import an XML configuration file, you need to use the `@ImportResource` annotation on your main configuration class or on any other configuration class. The `@ImportResource` annotation takes a list of XML configuration file paths as its value.

For example, suppose you have an XML configuration file named `myConfig.xml` in the `src/main/resources` directory of your application. You can import this configuration file into your main configuration class by using the `@ImportResource` annotation as follows:

```
@Configuration
@ImportResource("classpath:myConfig.xml")
public class MyAppConfig {
    // ...
}
```

In this example, the `myConfig.xml` file is imported into the `MyAppConfig` class, and

the beans defined in the myConfig.xml file will be available in the application context.

Configuration classes are special classes in a Spring Boot application that define the configuration of the application. You can import additional configuration classes by using the `@Import` annotation on the main configuration class or on any other configuration class. You can also import XML configuration files by using the `@ImportResource` annotation. When you import a configuration class or XML configuration file, all the beans defined in that class or file will be available in the application context.

4. Spring Beans and Dependency Injection

Overview of Spring Beans

In a Spring Boot application, a bean is an object that is managed by the Spring framework. Spring beans are typically defined in configuration classes and can be used throughout the application. Spring beans can be configured with properties and can have dependencies on other beans.

Dependency Injection

Dependency injection is a design pattern that is used to decouple objects and make them more flexible and maintainable. In a Spring Boot application, dependency injection is used to inject dependencies into beans at runtime. Dependency injection is achieved through constructor injection, setter injection, or field injection.

Constructor Injection

Constructor injection is a type of dependency injection that involves injecting dependencies through a constructor. In Spring Boot, constructor injection is achieved by defining a constructor in a bean and annotating the constructor parameters with the `@Autowired` annotation. When a bean is created, Spring will automatically

instantiate the bean's dependencies and pass them to the bean's constructor.

```
@Component

public class MyComponent {

    private MyDependency myDependency;

    @Autowired

    public MyComponent(MyDependency myDependency) {

        this.myDependency = myDependency;

    }

    // ...

}
```

Setter Injection

Setter injection is a type of dependency injection that involves injecting dependencies through setter methods. In Spring Boot, setter injection is achieved by defining a setter method in a bean and annotating the method with the `@Autowired` annotation. When a bean is created, Spring will automatically instantiate the bean's dependencies and call the bean's setter methods to set the dependencies.

```
@Component

public class MyComponent {

    private MyDependency myDependency;

    @Autowired

    public void setMyDependency(MyDependency myDependency) {

        this.myDependency = myDependency;

    }
```

```
// ...  
}
```

Field Injection

Field injection is a type of dependency injection that involves injecting dependencies directly into a bean's fields. In Spring Boot, field injection is achieved by annotating a bean's fields with the `@Autowired` annotation. When a bean is created, Spring will automatically instantiate the bean's dependencies and inject them into the bean's fields.

```
@Component  
  
public class MyComponent {  
  
    @Autowired  
    private MyDependency myDependency;  
  
    // ...  
}
```

Types of Dependency Injection

There are two types of dependency injection in Spring Boot:

- **Bean injection:** This is the process of injecting one bean into another bean. This is achieved through the use of the `@Autowired` annotation.
- **Value injection:** This is the process of injecting a value into a bean. This is achieved through the use of the `@Value` annotation.

Spring beans are objects that are managed by the Spring framework. Dependency injection is a design pattern that is used to decouple objects and make them more flexible and maintainable. In Spring Boot, dependency injection is achieved through constructor injection, setter injection, or field injection. There are two types of dependency injection in Spring Boot: bean injection and value injection.

